



DATA SCIENCE

Numpy for Beginners

The silent giant carrying your data science projects

Karlo Leskovar

Apr 12, 2021 14 min read

LATEST

EDITOR'S PICKS

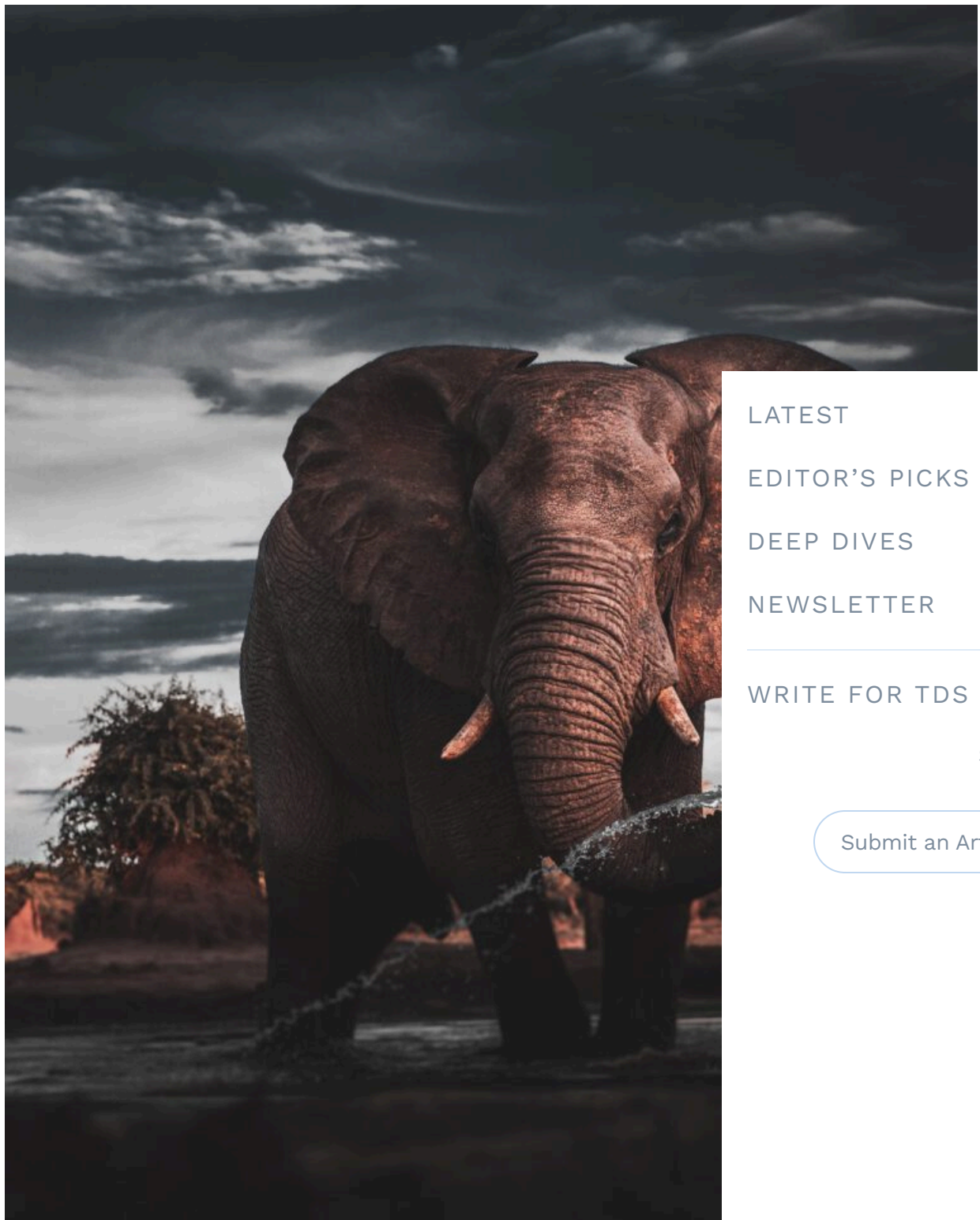
DEEP DIVES

NEWSLETTER

WRITE FOR TDS

[Sign in](#)

[Submit an Article](#)



LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Photo by [Geran de Klerk](#) on [Unsplash](#)

Until now we learned about Python programming which operators when to use and how to simplify tasks or make decisions using Control flow. Since (& Meteorology) we mostly work with a lot of n

need to look further into tools that can help us deal with a large amount of numbers. Therefore, this article is covering an incredibly popular library in data science circles, Numpy.

This article is structured as follows:

- Introduction
- Creating arrays
- Shape and Reshape
- Accessing elements and slicing an Array
- Maths and Statistics with Numpy
- Bonus content – Speed advantages of Numpy
- Conclusion

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Enjoy the learning!

Submit an Article

Introduction

Why to use Numpy? When printed, a Python list of floats, looks exactly the same as a **Numpy array**. But, mathematical operations on a bunch of numbers, statistical calculations and comparisons can get tedious. You could think Numpy is just a mathematical library with more functionality as lists, but is it? Let me explain.

The data in Numpy arrays is of homogeneous type, while the data in an array is of same type, while lists are pointers to objects, even though all the data is of same type. As a consequence, the Numpy arrays use

memory than regular lists. Also, most of the Numpy operations is implemented in the C language, meaning, the cost of Python loops and dynamic checking of the data type is avoided. This, yields a significant increase in processing speed when comparing Numpy to a Python list.

More often than one we encounter large datasets with tens of thousands rows of data, just think of hourly air temperature measurements for a county or region since the beginning in this region. If your weather service hourly air temperature for last 50 years, that's 000 rows of data, just for one station.

How to install Numpy?

Well, if using Anaconda, Numpy is preinstalled environment. However, more often than not, it to create new environments for new projects. Numpy, we run an Anaconda prompt and type:

```
conda install numpy
```

or

```
conda install -c anaconda numpy
```

If pip is being used, Numpy can be installed by

```
pip install numpy
```

How to import Numpy?

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

When importing certain libraries, including Numpy, we follow a convention, basically this means we use well established abbreviations for libraries. In the case of Numpy we use "**np**".

```
import numpy as np
```

The goal is that our code is reproducible, and every Python programmer in the World, knows what the following line does:

```
a = np.array([3,4])
```

Congrats, if you have imported Numpy, and used the `np.array()` command, you have successfully created your **array**. Let's see what happens if we print it out. We'll see something that looks like a list, but it's not. We'll see the type we see that's a "**numpy.ndarray**" (*n dimensional array*).

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

```
In [9]: a = np.array([3,4])
        print (a)
        print (type(a))

[3 4]
<class 'numpy.ndarray'>
```

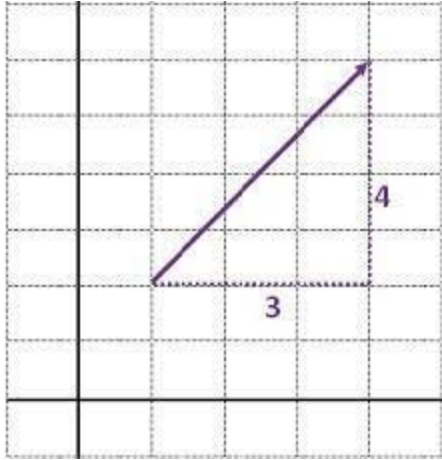
Image by author

Vectors?

In the example we saw how we can create an array. If you remember vectors from Math, well a 1-dimensional Numpy array is basically a vector. In our example, with the two numbers, 3 and 4, this vector lies in the 2D space.

space (geometric plane). It's same as in math when you had a vector:

$$v = 3i + 4j$$



taken from: <https://en.wikiversity.org/>

In Computer Science, vectors are just lists, where the length of the list (in our case, 2) is the number of dimensions of the vector. And in Data Science terms, a vector represents multiple features of an object. Think of meteorological measurements on Monday, you could measure temperature, precipitation, wind speed, snow, etc. To learn more about vectors, I highly recommend 3Blue1Brown.

Creating arrays

Above we already saw how to create a simple 1D NumPy array. Often, our data comes in more dimensions. For example, we have multiple features (like above), but also have measurements for multiple days in the week. In this case, we need to add a second dimension to our arrays. Let's see some

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

```
In [25]: weather_data = np.array([[13.4, 0.3, 1.6, 0], [14.8, 0., 1.1, 0]])
        weather_data
Out[25]: array([[13.4, 0.3, 1.6, 0. ],
               [14.8, 0. , 1.1, 0. ]])
```

image by author

To create a 2-D array, we provide a list, containing two lists. Think of this array as measurements on Monday (first list/list) and Tuesday (second row/list) where 1st column temperature, 2nd column precipitation, 3rd column wind speed and 4th column snow depth. The excel should clarify things.

	A	B	C	D	E
1		T_air	P	WS	SD
2	Monday	13.4	0.3	1.6	0
3	Tuesday	14.8	0	1.1	0
4					

image by author

Numpy also provides some useful functions to create arrays of zeros or ones. Try out the following commands and print out the results.

```
1 zeros = np.zeros([2,3])
2
3 ones = np.ones([3,4])
```

zeros_ones.py hosted with ❤ by GitHub

To demonstrate how to get the number of dimensions of a newly created array, I will use the **np.ones** function with the **ndim** attribute.

```
1 ones = np.ones([2,3,4,5])
```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

```
2 ones.ndim  
3  
4 4
```

dim_.py hosted with ❤️ by GitHub

[view raw](#)

So, our array has four dimensions, but how does a 4-dimensional array look like?

```
In [67]: print (ones)  
  
[[[ [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]]  
  
  [ [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]]  
  
  [ [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]  
      [1. 1. 1. 1. 1.]]]]
```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

[Sign in](#)

[Submit an Article](#)

Image by author

If we take a closer look, we can determine the dimension if we count the square brackets on end of the array also, a handy hack 😊

Another useful method is **arange**. It is used to get an evenly spaced array. We need to specify the **end number** (**int** or **float**).

```
1 range_a = np.arange(7)
2 range_a
3
4 array([0, 1, 2, 3, 4, 5, 6])
```

arange_1.py hosted with ❤ by GitHub

Numpy then assumes that starting point is zero. We need to provide the **starting** and **ending** point.

```
1 range_b = np.arange(7,12.)
2 range_b
3
4 array([ 7.,  8.,  9., 10., 11.])
```

arange_2.py hosted with ❤ by GitHub

And, we can specify the **step** as follows:

```
1 range_c = np.arange(7,12,2)
2 range_c
3
4 array([ 7,  9, 11])
```

arange_3.py hosted with ❤ by GitHub

Similarly, with the method **linspace** we can create an array of evenly spaced elements but instead of the step, **linspace** takes the number of elements in the array. Here we create an array of 5 elements between 7 and 12. Also, contrary to most of Python methods, the last number (end) is **including**.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

```
1 range_d = np.linspace(7,12,5)
2 range_d
3
4 array([ 7. ,  8.25,  9.5 , 10.75, 12. ])
```

linspace.py hosted with ❤ by GitHub

[view raw](#)

Shape and Reshaping

Before, we checked how many dimensions (or array had. But what if we are interested in how elements are in each of the dimensions? The `shape` attribute comes in handy. Since we have 4 dimension, we get 4 numbers.

```
In [69]: ones.shape
Out[69]: (2, 3, 4, 5)
```

Image by author

To count the number of elements in the whole array, we use the `size` attribute.

```
In [70]: ones.size
Out[70]: 120
```

Image by author

In order to change the shape of an array, we use the `reshape()` method. Care has to be taken though that the reshaped array has to be of same size as the original array. We will explain..

[LATEST](#)

[EDITOR'S PICKS](#)

[DEEP DIVES](#)

[NEWSLETTER](#)

[WRITE FOR TDS](#)

[Sign in](#)

[Submit an Article](#)

```
In [77]: zeros_new = zeros.reshape(3,2)
print (zeros, zeros.shape)
print(zeros_new, zeros_new.shape)

[[0. 0. 0.]
 [0. 0. 0.]] (2, 3)
[[0. 0.]
 [0. 0.]
 [0. 0.]] (3, 2)
```

Image by author

The original zeros array had the shape of (2, 3) and we can reshape it into (3,2), (6, 1) or (1,6), since it has 6 elements. I shall mention, that in case of reshaping to (1, 6) we change the number of dimensions, from a 2-D array, to a 1-D array, but as long we take care of the shape, we are on the safe side.

A handy "shortcut" to a 1-D array are the **flatten** methods. The difference is that **flatten** creates a copy of the original array, while **ravel** creates a reference to the original array. So, using **ravel()** has the consequence that if you change for example some of the data in the new array while also change the data in the original array.

The usage depends on the specific task, most of the time we use the **flatten()** method.

```
In [79]: ones_flat = ones.flatten()
print (ones.shape)
print (ones_flat.shape)

(2, 3, 4, 5)
(120,)
```

Image by author

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Last but not least, let's not forget the `transpose()` method. This method simply swaps the rows and columns of an array.

```
In [85]: zeros_transposed = zeros.T
print (zeros_transposed, zeros_transposed.shape)

[[0. 0.]
 [0. 0.]
 [0. 0.]] (3, 2)
```

Image by author

In this case, the result is same as before with of a multidimensional array, all dimensions get see.

```
In [87]: ones_transposed = ones.T
print (ones_transposed.shape)

(5, 4, 3, 2)
```

Image by author

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Accessing elements and slicing an

Until now, we saw how to create, find proportion or flatten an array. Let's turn our focus now on from an array using **indexing** and **slicing**. To **slicing** means to access its elements by providing the elements **index**.

The default syntax of slicing involves the array square brackets, like for Python lists, as follow

`array_name[start_index : end_index : step_s`

```
In [32]: hourly_air_temp = [0, 1, 1, 2, 3, 3, 4, 6, 7, 9, 9, 10,
                             12, 14, 16, 16, 14, 11, 9, 8, 7, 4, 3, 2]

print ("Temperature very 2 hours - afternoon:", hourly_air_temp[11:20:2])

Temperature very 2 hours - afternoon: [10, 14, 16, 11, 8]
```

Image by author

In our array the temperature measurements start from 1:00. To clarify the things, we printed out hourly temperatures at 12:00, 14:00, 16:00, 18:00 and 20:00.

LATEST

If we don't define the step size, every element range get returned. For example if we need the temperatures from 7:00 to 12:00.

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

```
In [35]: print ("Temperatures from 7:00 to 12:00 - morning:",
               Temperatures from 7:00 to 12:00 - morning: [4, 6, 7,
```

Image by author

WRITE FOR TDS

Sign in

Submit an Article

As usual in Lists, so in Numpy also, the start_index including, while the end_index is not including index in an array is always zero. So to access temperature at 7:00, we input 6-th index. And since we need measurements until 12 (which is at index no. 11) 12th index (it's excluding).

Slicing 2-D arrays

When slicing a 2-D array, we need to specify the column of the element we desire. It can be a little first, but when tried on few examples, it's soon easy.

The syntax to slice a 2-D array is as follows:

```
array_name[row_start_index : row_end_index :  
row_step_size, column_start_index : column_end_index :  
column_step_size]
```

To see how slicing a 2-D array works, I will first extend our weather_data array to a full week, so we shall get an array of shape (7,4).

```
In [39]: weather_data = np.array([[13.4, 0.3, 1.6, 0],  
                                  [14.8, 0. , 1.1, 0],  
                                  [14.9, 0.1, 1.2, 0],  
                                  [13.2, 8.6, 3.7, 0],  
                                  [15.7, 0. , 0.4, 0],  
                                  [15.8, 0. , 0.8, 0],  
                                  [16.4, 0. , 0. , 0]])  
  
weather_data.shape  
  
Out[39]: (7, 4)
```

Image by author

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Let's say we want to know the weekly precipitation. We need to slice out all rows, and the 2nd column

```
In [46]: weekly_precip = weather_data[:,1]  
weekly_precip  
  
Out[46]: array([0.3, 0. , 0.1, 8.6, 0. , 0. , 0. ])
```

Image by author

In this case, to select all the rows we use a **colon** to select the 2nd column we use the **index value** (**indices start from zero**).

Feel free to try out other possibilities, I would like to share with you some rules that, with integrals in Math, there are certain rules that

practice makes perfect.

Negative slicing is also allowed, and works in same manner as with python lists. The last item in an array has the index of -1.

"Finding" data in an array

Another way of finding data in an array is by using a very popular function inside of Numpy, **np.where()**. It returns the indices of elements that meet a condition. Commonly, it's used when finding elements that are equal or less than a number. The basic syntax is as follows:

```
np.where(condition [, x, y])
```

x and **y** are parameters which can be used to provide a value in the array that meets the given condition. **don't provide x and y** (we just need to find indices that meet the condition), or we **provide both x and y** (values at the found index get changed by **x** if **condition is True** and **y** if **condition is False**). Very similar like **IF()** function.

Let's say we want to print out the indices of data that is warmer than 14.5 degrees °C.

```
In [6]: print ("Warmer than 14.5 °C: ", np.where(weather_data['temp'] > 14.5)[0])
Warmer than 14.5 °C: (array([1, 2, 4, 5, 6], dtype=int64),)
```

Image by author

We have two important things here: firstly, we learned slicing to select the first column (all rows).

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

search all weekdays), and then we set the condition > 14.5 .

Let's say we want to convert all temperature values greater than 14.5 to Fahrenheit degrees, and store the resulting array to the variable **weather_b**.

```
In [11]: weather_b = np.where(weather_data[:,0] > 14.5, weather_data[:,0]*1.8+32, weather_data[:,0])
         weather_b
Out[11]: array([13.4 , 58.64, 58.82, 13.2 , 60.26, 60.44, 61.52])
```

Image by author

Again, we first provide a condition (>14.5 °C), then we provide what value (multiply the value by 1.8 and add 32) if the condition is **True**, and what value (use the existing value) if the condition is **False**. Notice, that we always slice the array, since we only want to change the first column.

We can now create a new array called **weather_f** (a copy of **weather_data**) but the temperature values will now be in Fahrenheits.

```
In [19]: weather_f = weather_data.copy()
         weather_f[:,0] = weather_b
         weather_f
Out[19]: array([[13.4 , 0.3 , 1.6 , 0. ],
                [58.64, 0. , 1.1 , 0. ],
                [58.82, 0.1 , 1.2 , 0. ],
                [13.2 , 8.6 , 3.7 , 0. ],
                [60.26, 0. , 0.4 , 0. ],
                [60.44, 0. , 0.8 , 0. ],
                [61.52, 0. , 0. , 0. ]])
```

Image by author

First, we make a copy of **weather_data** (remember the **copy()** and **ravel()** methods), to avoid changes in original **weather_data** array, and then we slice the new

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

column), and replace the values with the ones calculated in weather_b.

Maths and Statistics with Numpy

Finally we've come to my favourite part of Numpy, mathematical and statistical operations. This is what makes Numpy so great, and superior to standard Python with common lists. It's the simplicity and speed when dealing with a great amount of numerical data. I first take a look to mathematical operations. I'll start with an example with division, but the general syntax is the same for other operations, and can be looked up at the [pages](#).

Our weather_data array contains precipitation millimetres, lets convert those to metres. To convert millimetres to metres, we need to divide the v

```
In [16]: precip_metres = weather_data[:, 1]/1000
precip_metres
```

```
Out[16]: array([0.0003, 0.      , 0.0001, 0.0086, 0.      , 0.      , 0.0001,
```

Image by author

Again we use slicing, to select the second column and divide the values by 1000. For practice, try precipitation values from the array `weather_fv` converted to metres. (you can do the change on array directly)

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

[Submit an Article](#)

As for the statistics example, I shall use the most common case, we need to calculate the average temperature, precipitation, wind speed and snow depth values for the week. The Numpy function to calculate the average values is called **np.mean()**. The basic syntax of **np.mean()** function is as follows:

```
np.mean(a, axis=None, dtype=None, out=None, keepdims=False, *, where=)
```

For our case, the important part is the axis. Since we want to calculate the mean values for each column, we set the axis parameter to 0. Setting the axis to 1, we would get the result row-wise.

```
In [24]: mean_values = np.mean(weather_data, axis=0)
          mean_values
Out[24]: array([14.88571429,  1.28571429,  1.25714286,  0.0])
```

Image by author

Other statistical functions retain an equal or similar syntax and can be looked up at the [Statistics section](#) of the Numpy site.

Bonus content – Speed advantages

I've mentioned that Numpy also has some speed advantages. This probably got you tempted. Let's see that in a practical example. Is Numpy really that faster than a for loop?

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

First, we will create an random array of floats, let's say its hourly air temperature measured at some location in th US. The length of the array is 30 000.

```
In [57]: hourly_temp_F = np.random.uniform(low=-20., high=100., size=30000)
len(hourly_temp_F)
hourly_f_list = hourly_temp_F.tolist()
```

Image by author

We want to convert those numbers to Celsius measure the time needed using a for loop and then using Numpy.

```
In [60]: %%time
hourly_temp_C = list()
for temp in hourly_f_list:
    hourly_temp_C.append((temp-32)*(5/9))
Wall time: 4.99 ms

In [61]: %%time
hourly_temp_C = (hourly_temp_F-32)*(5/9)
Wall time: 997 µs
```

Image by author

So, the time taken using an loop over a Pythor around 5 ms, while using an Numpy array the : took less than 1 ms. So Numpy is in this specifi around 5 times faster.

Also, please consider that this test is not com applicable, it really depends on the speed of y (mostly CPU) and the chosen task. Since this is topic of the article, I'll leave it to you to check

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

that are covering speed benefits of Numpy, and hopefully, try it out yourself, on your specific task with your data.

Conclusion

Congrats, we have covered the basics of Numpy. But please, keep in mind that Numpy is much more than this article. Practice is very important. Here, I've presented functions that I use very often in my tasks and there is a lot more of functions and methods in

Make sure you check the official Numpy site for possibilities that may be more related to your tasks. Don't get discouraged at first, all of these seem a little complex at first, but believe me, think, you will adopt them in no time. 😊

My next article will cover another popular library for Science applications, called Pandas. Stay tuned, keep practicing, because, practice makes

For any questions or suggestions regarding this or other articles on [Medium](#), feel free to contact

Thank you for taking the time, Cheers! 😊

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

WRITTEN BY

Karlo Leskovar

See all from Karlo Leskovar

Data Science

Numpy

Python

Statistics

Weather

Share This Article



Towards Data Science is a community platform for data science professionals to share their insights and knowledge. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

LATEST

EDITOR'S PICKS

DEEP DIVES

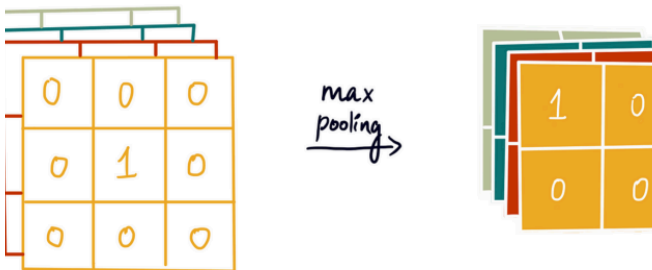
NEWSLETTER

WRITE FOR TDS

Sign in

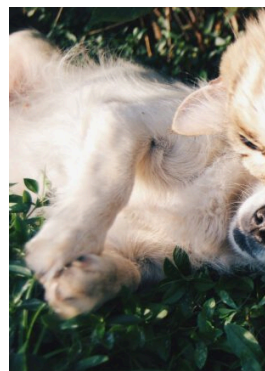
Submit an Article

Related Articles



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow



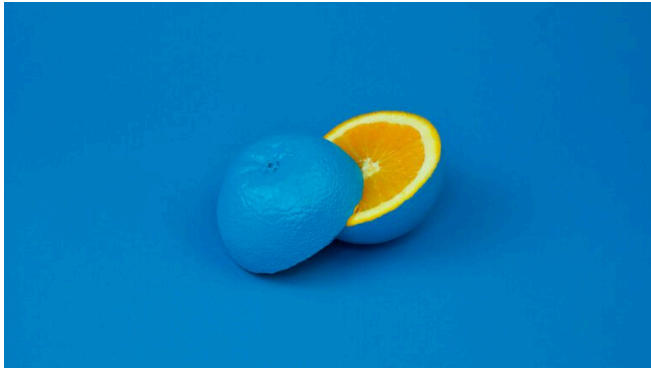
ARTIFICIAL INTELLIGENCE

How to Forecast Time Series

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read



DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of...

Piero Paialunga

August 21, 2024 12 min read



DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read

A beginner's guide to forecast reconciliation

Dr. Robert Kübler

August 20, 2024 13 min read

DATA SCIENCE

Solving a Constrained Project Scheduling Problem with Quantum Annealing

Solving the resource scheduling problem with Wave's hybrid combinatorial model (CQM)

Luis Fernando PÉREZ
August 20, 2024

LATEST

EDITOR'S PICKS

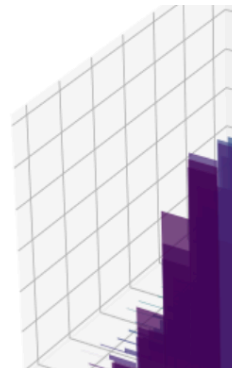
DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article



DATA SCIENCE

Must-Know in Bivariate Normal Explained

Derivation and practical applications of a powerful concept

Luigi Battistoni

August 14, 2024 7 min read



DATA SCIENCE

Our Columns

Columns on TDS are carefully curated collections of posts on a particular idea or category...

TDS Editors

November 14, 2020 4 min read

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in



Submit an Article

Your home for data science and AI. The world's leading publication for data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2026

Subscribe to Our Newsletter

WRITE FOR TDS · ABOUT · ADVERTISE · PRIVACY POLICY